Artificial Intelligence and Soft Computing

Practical Programs (Python)

EXP01 - EXP08 + Additional Basic Programs | Mumbai University

EXP01	N-Queen using Uninformed Search
EXP02	Informed Search using A* Algorithm
EXP03	Adversarial Search
EXP04	Predicate Logic
EXP05	Kohonen Self-Organizing Map
EXP06	Multi-Layer Perceptron (Supervised Learning)
EXP07	Student Performance Prediction using EBPTA
EXP08	Fuzzy Membership Functions
EXTRA	Additional Basic Programs (DFS, BFS and more)

Name: _____ Roll No: _____ Div: _____

All sample outputs were produced by executing the programs (Python 3.13.18).

EXP01: N-Queen using Uninformed Search

Aim: Implement N-Queen Uninformed Search Technique.

Note: the practical file lists 'Code 3' (TSP) twice and has no 'Code 2'; it is included once.

Code 1: 8-Queen Problem (display one valid solution)

Program

```
N = 8

def is_safe(board, row, col):
    # check column and both diagonals against queens already placed
    for r in range(row):
        if board[r] == col or abs(board[r] - col) == abs(r - row):
            return False
    return True

def solve(board, row):
    if row == N:
        return True
    for col in range(N):
        if is_safe(board, row, col):
            board[row] = col
            if solve(board, row + 1):
                return True
            board[row] = -1
    return False

board = [-1] * N
if solve(board, 0):
    print("One valid solution of the 8-Queen problem")
    print("Queen positions (row, col):", [(r, board[r]) for r in range(N)])
    print()
    for r in range(N):
        print(" ".join("Q" if board[r] == c else "." for c in range(N)))
else:
    print("No solution exists")
```

Sample Output

```
One valid solution of the 8-Queen problem
Queen positions (row, col): [(0, 0), (1, 4), (2, 7), (3, 5), (4, 2), (5, 6), (6, 1), (7, 3)]

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Result: Backtracking (DFS) places 8 queens so that no two queens attack each other.

Code 3: TSP for 4 cities using DFS (minimum cost tour)

Program

```
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]
cities = ["A", "B", "C", "D"]
n = len(graph)
best_cost = float("inf")
best_path = []

def dfs(city, visited, path, cost):
    global best_cost, best_path
    if len(path) == n:
        total = cost + graph[city][0]          # return to starting city
        tour = path + [0]
        print("Tour:", " -> ".join(cities[i] for i in tour), "| Cost =", total)
        if total < best_cost:
            best_cost, best_path = total, tour
        return
    for nxt in range(n):
        if nxt not in visited:
            dfs(nxt, visited + [nxt], path + [nxt], cost + graph[city][nxt])

print("All tours found by DFS:")
dfs(0, [0], [0], 0)
print("\nMinimum cost tour:", " -> ".join(cities[i] for i in best_path))
print("Minimum cost:", best_cost)
```

Sample Output

```
All tours found by DFS:
Tour: A -> B -> C -> D -> A | Cost = 95
Tour: A -> B -> D -> C -> A | Cost = 80
Tour: A -> C -> B -> D -> A | Cost = 95
Tour: A -> C -> D -> B -> A | Cost = 80
Tour: A -> D -> B -> C -> A | Cost = 95
Tour: A -> D -> C -> B -> A | Cost = 95

Minimum cost tour: A -> B -> D -> C -> A
Minimum cost: 80
```

Result: DFS explores every tour from city A; the tour with the minimum cost is reported.

EXP02: Informed Search using A* Algorithm

Aim: Implement Informed Search Technique for 15-Puzzle Problem using A* Algorithm.

Code 1: 15-Puzzle using A*

Program

```
import heapq

GOAL = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0)

def heuristic(state):
    # sum of Manhattan distances of all tiles from their goal positions
    dist = 0
    for i, tile in enumerate(state):
        if tile != 0:
            goal_row, goal_col = divmod(tile - 1, 4)
            dist += abs(i // 4 - goal_row) + abs(i % 4 - goal_col)
    return dist

def neighbors(state):
    z = state.index(0)
    row, col = divmod(z, 4)
    for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
        r, c = row + dr, col + dc
        if 0 <= r < 4 and 0 <= c < 4:
            new = list(state)
            n = r * 4 + c
            new[z], new[n] = new[n], new[z]
            yield tuple(new)

def a_star(start):
    heap = [(heuristic(start), 0, start, [start])]    # (f, g, state, path)
    visited = set()
    while heap:
        f, g, state, path = heapq.heappop(heap)
        if state == GOAL:
            return path
        if state in visited:
            continue
        visited.add(state)
        for nxt in neighbors(state):
            if nxt not in visited:
                h = heuristic(nxt)
                heapq.heappush(heap, (g + 1 + h, g + 1, nxt, path + [nxt]))
    return None

start = (1, 2, 3, 4, 5, 0, 7, 8, 9, 6, 10, 12, 13, 14, 11, 15)
path = a_star(start)
print("Number of moves:", len(path) - 1)
for step, state in enumerate(path):
    print("\nStep", step, "(h =", heuristic(state), ")")
    for i in range(0, 16, 4):
        print(*[f"{t:2}" if t else " _" for t in state[i:i + 4]])
```

Sample Output

```
Number of moves: 4

Step 0 (h = 4 )
 1  2  3  4
 5  _  7  8
 9  6 10 12
13 14 11 15

Step 1 (h = 3 )
 1  2  3  4
 5  6  7  8
 9  _ 10 12
13 14 11 15

Step 2 (h = 2 )
 1  2  3  4
 5  6  7  8
 9 10  _ 12
13 14 11 15

Step 3 (h = 1 )
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14  _ 15

Step 4 (h = 0 )
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15  _
```

Result: A* with the Manhattan-distance heuristic reaches the goal in the minimum number of moves.

Code 2: Water Jug Problem using A*

Program

```
import heapq

CAP_A, CAP_B, TARGET = 4, 3, 2

def heuristic(x, y):
    return min(abs(x - TARGET), abs(y - TARGET))

def next_moves(x, y):
    pour_ab = min(x, CAP_B - y)
    pour_ba = min(y, CAP_A - x)
    return [
        ("Fill A", (CAP_A, y)),
        ("Fill B", (x, CAP_B)),
        ("Empty A", (0, y)),
        ("Empty B", (x, 0)),
        ("Pour A -> B", (x - pour_ab, y + pour_ab)),
        ("Pour B -> A", (x + pour_ba, y - pour_ba)),
    ]

def a_star():
    start = (0, 0)
    heap = [(heuristic(*start), 0, start, [("Start", start)])]
    visited = set()
    while heap:
        f, g, (x, y), path = heapq.heappop(heap)
        if x == TARGET or y == TARGET:
            return path
        if (x, y) in visited:
            continue
        visited.add((x, y))
        for action, state in next_moves(x, y):
            if state not in visited:
                h = heuristic(*state)
                heapq.heappush(heap, (g + 1 + h, g + 1, state, path + [(action, state)]))
    return None

path = a_star()
print("Jug A = 4 L, Jug B = 3 L, Target = 2 L\n")
print(f"{'Step':<6}{'Action':<14}{'(A, B)'}")
for i, (action, state) in enumerate(path):
    print(f"{i:<6}{action:<14}{state}")
print("\nTotal steps:", len(path) - 1)
```

Sample Output

```
Jug A = 4 L, Jug B = 3 L, Target = 2 L

Step  Action        (A, B)
0     Start         (0, 0)
1     Fill B        (0, 3)
2     Pour B -> A   (3, 0)
3     Fill B        (3, 3)
4     Pour B -> A   (4, 2)

Total steps: 4
```

Result: A* finds the shortest sequence of states that measures 2 litres using 4 L and 3 L jugs.

Code 3: N-Queen using A*

Program

```
import heapq

N = 8

def is_safe(cols, col):
    row = len(cols)
    return all(c != col and abs(c - col) != row - r for r, c in enumerate(cols))

def a_star():
    heap = [(N, N, [])]            # (f = g + h, h, columns placed so far)
    expanded = 0
    while heap:
        f, h, cols = heapq.heappop(heap)
        expanded += 1
        if h == 0:
            return cols, expanded
        for col in range(N):
            if is_safe(cols, col):
                g = len(cols) + 1
                heapq.heappush(heap, (g + (N - g), N - g, cols + [col]))
    return None, expanded

solution, expanded = a_star()
print("A* solution for", N, "queens (column of each row):", solution)
print("Nodes expanded:", expanded)
print()
for r in range(N):
    print(" ".join("Q" if solution[r] == c else "." for c in range(N)))
```

Sample Output

```
A* solution for 8 queens (column of each row): [0, 4, 7, 5, 2, 6, 1, 3]
Nodes expanded: 114

Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Result: A* (f = g + h, h = queens still to be placed) finds a valid 8-Queen placement.

EXP03: Adversarial Search

Aim: Implement Alpha-Beta Pruning. Implement MIN-MAX.

Part 1: MIN-MAX

Program

```
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
DEPTH = 3

def minimax(depth, index, is_max):
    if depth == DEPTH:
        return leaves[index]
    left = minimax(depth + 1, index * 2, not is_max)
    right = minimax(depth + 1, index * 2 + 1, not is_max)
    value = max(left, right) if is_max else min(left, right)
    kind = "MAX" if is_max else "MIN"
    pad = "  " * depth
    print(f"{pad}{kind} node (level {depth}): children ({left}, {right}) -> {value}")
    return value

print("Leaf values:", leaves, "\n")
best = minimax(0, 0, True)
print("\nOptimal value at the root (MAX player):", best)
```

Sample Output

```
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]

    MAX node (level 2): children (3, 5) -> 5
    MAX node (level 2): children (6, 9) -> 9
  MIN node (level 1): children (5, 9) -> 5
    MAX node (level 2): children (1, 2) -> 2
    MAX node (level 2): children (0, -1) -> 0
  MIN node (level 1): children (2, 0) -> 0
MAX node (level 0): children (5, 0) -> 5

Optimal value at the root (MAX player): 5
```

Result: The MIN-MAX value of the root of the game tree (depth 3, 8 leaves) is computed.

Part 2: Alpha-Beta Pruning

Program

```
import math

leaves = [3, 5, 6, 9, 1, 2, 0, -1]
DEPTH = 3
checked = 0

def alphabeta(depth, index, is_max, alpha, beta):
    global checked
    if depth == DEPTH:
        checked += 1
        return leaves[index]
    best = -math.inf if is_max else math.inf
    for k in (0, 1):
        value = alphabeta(depth + 1, index * 2 + k, not is_max, alpha, beta)
        if is_max:
            best = max(best, value)
            alpha = max(alpha, best)
        else:
            best = min(best, value)
            beta = min(beta, best)
        if beta <= alpha:
            if k == 0:
                kind = "MAX" if is_max else "MIN"
                print(f"{'  ' * depth}Pruned at {kind} node: alpha={alpha}, beta={beta}")
            break
    return best

print("Leaf values:", leaves, "\n")
best = alphabeta(0, 0, True, -math.inf, math.inf)
print("\nOptimal value at the root (MAX player):", best)
print("Leaf nodes evaluated:", checked, "out of", len(leaves))
```

Sample Output

```
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]

    Pruned at MAX node: alpha=6, beta=5
  Pruned at MIN node: alpha=5, beta=2

Optimal value at the root (MAX player): 5
Leaf nodes evaluated: 5 out of 8
```

Result: Same optimal value as MIN-MAX, but fewer leaves are evaluated because branches are pruned.

EXP04: Predicate Logic

Aim: Implement Predicate Logic (Library Management System).

Library Management System using predicates

Program

```
books = {"B1": "Python", "B2": "AI", "B3": "DBMS", "B4": "Networks"}
members = ["Ravi", "Sita", "Amit"]
issued = {"B1": "Ravi", "B3": "Sita"}          # facts: Issued(book, member)

# predicates
def Book(b):
    return b in books

def Member(m):
    return m in members

def Issued(b, m):
    return issued.get(b) == m

def Available(b):
    return Book(b) and b not in issued

def CanIssue(m, b):                              # rule: Member(m) and Available(b)
    return Member(m) and Available(b)

def issue(m, b):
    if CanIssue(m, b):
        issued[b] = m
        print(f"{b} ({books[b]}) issued to {m}")
    else:
        print(f"Cannot issue {b} to {m}")

print("FACTS")
print("Books   :", books)
print("Members :", members)
print("Issued  :", issued)

print("\nQUERIES")
print("Available(B2)       =", Available("B2"))
print("Issued(B1, Ravi)    =", Issued("B1", "Ravi"))
print("CanIssue(Amit, B2)  =", CanIssue("Amit", "B2"))
print("CanIssue(Amit, B1)  =", CanIssue("Amit", "B1"))
print("exists b : Available(b) =", any(Available(b) for b in books))
print("forall b : Available(b) =", all(Available(b) for b in books))
print("Books issued to Ravi    =", [b for b in books if Issued(b, "Ravi")])

print("\nACTIONS")
issue("Amit", "B2")
issue("Amit", "B1")
print("Available books now:", [books[b] for b in books if Available(b)])
```

Sample Output

```
FACTS
Books   : {'B1': 'Python', 'B2': 'AI', 'B3': 'DBMS', 'B4': 'Networks'}
Members : ['Ravi', 'Sita', 'Amit']
Issued  : {'B1': 'Ravi', 'B3': 'Sita'}

QUERIES
Available(B2)       = True
Issued(B1, Ravi)    = True
CanIssue(Amit, B2)  = True
CanIssue(Amit, B1)  = False
exists b : Available(b) = True
forall b : Available(b) = False
Books issued to Ravi    = ['B1']

ACTIONS
B2 (AI) issued to Amit
Cannot issue B1 to Amit
Available books now: ['Networks']
```

Result: Facts, rules and quantified queries (exists / for all) are evaluated correctly.

EXP05: Kohonen Self-Organizing Map

Aim: Implement a Simple Kohonen Self-Organizing Map.

Kohonen SOM (4 input vectors, 2 cluster units)

Program

```
import numpy as np

X = np.array([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 1, 1]], dtype=float)
W = np.array([[0.2, 0.6, 0.5, 0.9], [0.8, 0.4, 0.7, 0.3]])   # 2 cluster units
alpha = 0.6                                                    # learning rate
epochs = 10

def winner(x):
    distances = ((W - x) ** 2).sum(axis=1)       # squared Euclidean distance
    return int(np.argmin(distances))

print("Initial weights:\n", W)
for epoch in range(1, epochs + 1):
    for x in X:
        j = winner(x)
        W[j] += alpha * (x - W[j])               # update winning unit only
    alpha *= 0.5
    if epoch <= 3 or epoch == epochs:
        print(f"\nEpoch {epoch}: weights (next learning rate = {alpha:.4f})")
        print(W.round(3))

print("\nClustering result")
for x in X:
    print("Input", x.astype(int), "-> Cluster", winner(x))
```

Sample Output

```
Initial weights:
 [[0.2 0.6 0.5 0.9]
 [0.8 0.4 0.7 0.3]]

Epoch 1: weights (next learning rate = 0.3000)
[[0.032 0.096 0.68  0.984]
 [0.968 0.304 0.112 0.048]]

Epoch 2: weights (next learning rate = 0.1500)
[[0.016 0.047 0.633 0.992]
 [0.984 0.359 0.055 0.024]]

Epoch 3: weights (next learning rate = 0.0750)
[[0.011 0.034 0.607 0.994]
 [0.989 0.387 0.04  0.017]]

Epoch 10: weights (next learning rate = 0.0006)
[[0.008 0.025 0.582 0.996]
 [0.992 0.413 0.029 0.013]]

Clustering result
Input [1 1 0 0] -> Cluster 1
Input [0 0 0 1] -> Cluster 0
Input [1 0 0 0] -> Cluster 1
Input [0 0 1 1] -> Cluster 0
```

Result: The map self-organizes: similar input vectors are assigned to the same cluster unit.

EXP06: Multi-Layer Perceptron (Supervised Learning)

Aim: Implement a supervised learning algorithm using Multi-Layer Perceptron (MLP) and predict the output for given input data.

MLP learning the XOR function

Program

```
import numpy as np
from sklearn.neural_network import MLPClassifier

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 1, 1, 0])                        # XOR truth table

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation="tanh",
    solver="lbfgs",
    max_iter=1000,
    random_state=1,
)
model.fit(X, y)

pred = model.predict(X)
prob = model.predict_proba(X)[:, 1]
print("MLP: 2 inputs -> 4 hidden neurons (tanh) -> 1 output\n")
print("x1  x2  Target  Predicted  Probability")
for (x1, x2), t, p, pr in zip(X, y, pred, prob):
    print(f"{x1}   {x2}     {t}        {p}        {pr:.3f}")
print("\nTraining accuracy:", model.score(X, y) * 100, "%")
print("Prediction for input [1, 0]:", model.predict([[1, 0]])[0])
```

Sample Output

```
MLP: 2 inputs -> 4 hidden neurons (tanh) -> 1 output

x1  x2  Target  Predicted  Probability
0   0     0        0        0.000
0   1     1        1        0.999
1   0     1        1        1.000
1   1     0        0        0.001

Training accuracy: 100.0 %
Prediction for input [1, 0]: 1
```

Result: The trained MLP predicts the correct XOR output for every input.

EXP07: Student Performance Prediction using EBPTA

Aim: A college wants to predict whether a student will Pass or Fail based on Hours Studied and Attendance, using the Error Back Propagation Training Algorithm (EBPTA).

EBPTA: 2-3-1 network, sigmoid activation

Program

```
import numpy as np

np.random.seed(1)
# columns: hours studied, attendance (%)
data = np.array([
    [1, 40], [2, 50], [3, 60], [4, 55],
    [6, 75], [7, 80], [8, 90], [9, 95],
])
target = np.array([[0], [0], [0], [0], [1], [1], [1], [1]])     # 1 = Pass, 0 = Fail
X = data / [10, 100]                                             # normalise inputs

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

lr = 0.5
W1 = np.random.randn(2, 3)
b1 = np.zeros(3)
W2 = np.random.randn(3, 1)
b2 = np.zeros(1)

for epoch in range(1, 5001):
    # forward pass
    H = sigmoid(X @ W1 + b1)
    O = sigmoid(H @ W2 + b2)
    # backward pass: error terms (delta) and weight updates
    delta_o = (target - O) * O * (1 - O)
    delta_h = (delta_o @ W2.T) * H * (1 - H)
    W2 += lr * H.T @ delta_o
    b2 += lr * delta_o.sum(axis=0)
    W1 += lr * X.T @ delta_h
    b1 += lr * delta_h.sum(axis=0)
    if epoch % 1000 == 0:
        print(f"Epoch {epoch:4d}   Mean squared error = {np.mean((target - O) ** 2):.5f}")

def predict(hours, attendance):
    x = np.array([hours, attendance]) / [10, 100]
    return sigmoid(sigmoid(x @ W1 + b1) @ W2 + b2)[0]

print("\nTraining data results")
print("Hours  Attendance  Output  Result")
for (hrs, att), t in zip(data, target):
    p = predict(hrs, att)
    print(f"{hrs:>5}  {att:>10}  {p:.3f}   {'Pass' if p > 0.5 else 'Fail'}")

print("\nPrediction for new students")
for hrs, att in [(7, 85), (2, 45), (8, 92), (1, 35)]:
    p = predict(hrs, att)
    result = "Pass" if p > 0.5 else "Fail"
    print(f"Hours = {hrs}, Attendance = {att}%  ->  {p:.3f}  ->  {result}")
```

Sample Output

```
Epoch 1000   Mean squared error = 0.00162
Epoch 2000   Mean squared error = 0.00063
Epoch 3000   Mean squared error = 0.00037
Epoch 4000   Mean squared error = 0.00026
Epoch 5000   Mean squared error = 0.00020

Training data results
Hours  Attendance  Output  Result
    1          40  0.000   Fail
    2          50  0.001   Fail
    3          60  0.005   Fail
    4          55  0.027   Fail
    6          75  0.972   Pass
    7          80  0.996   Pass
    8          90  0.999   Pass
    9          95  1.000   Pass

Prediction for new students
Hours = 7, Attendance = 85%  ->  0.997  ->  Pass
Hours = 2, Attendance = 45%  ->  0.000  ->  Fail
Hours = 8, Attendance = 92%  ->  0.999  ->  Pass
Hours = 1, Attendance = 35%  ->  0.000  ->  Fail
```

Result: The network is trained with error back propagation and predicts Pass/Fail for new students.

EXP08: Fuzzy Membership Functions

Aim: Analyze and implement Triangular, Trapezoidal and Gaussian membership functions to classify room temperature (0-40 C) as Cold, Comfortable or Hot, and compare membership values for 10, 15, 20, 25, 30, 35 C. Exercise 1: classify attendance (0-100%) as Low, Moderate or High.

Room temperature: Cold / Comfortable / Hot (values and graphs)

Program

```
import numpy as np
import matplotlib.pyplot as plt

def triangular(x, a, b, c):
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

def trapezoidal(x, a, b, c, d):
    return np.maximum(np.minimum(np.minimum((x - a) / (b - a), 1), (d - x) / (d - c)), 0)

def gaussian(x, mean, sigma):
    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# parameters of each set: (triangular), (trapezoidal), (gaussian)
sets = {
    "Cold": ((-20, 0, 20), (-10, 0, 10, 20), (5, 6)),
    "Comfortable": ((10, 20, 30), (10, 18, 24, 32), (22, 5)),
    "Hot": ((20, 40, 60), (22, 32, 40, 50), (38, 6)),
}
functions = (triangular, trapezoidal, gaussian)
names = ("Triangular", "Trapezoidal", "Gaussian")

for i, name in enumerate(names):
    print(f"\n{name} membership values")
    print(f"{'Temp':>6}{'Cold':>8}{'Comfortable':>14}{'Hot':>8}   Class")
    for t in (10, 15, 20, 25, 30, 35):
        mu = [functions[i](t, *sets[s][i]) for s in sets]
        label = list(sets)[int(np.argmax(mu))]
        print(f"{t:>4} C{mu[0]:>8.2f}{mu[1]:>14.2f}{mu[2]:>8.2f}   {label}")

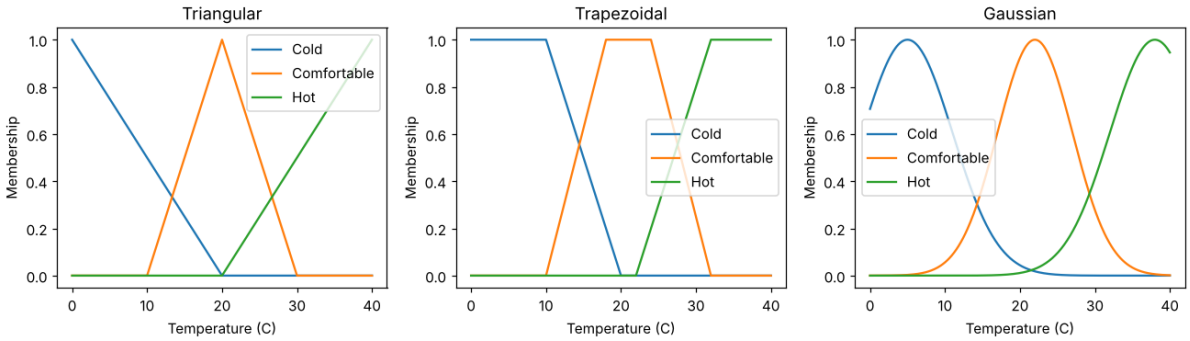
x = np.linspace(0, 40, 401)
for i, name in enumerate(names):
    plt.subplot(1, 3, i + 1)
    for s, p in sets.items():
        plt.plot(x, functions[i](x, *p[i]), label=s)
    plt.title(name)
    plt.xlabel("Temperature (C)")
    plt.ylabel("Membership")
    plt.legend()
plt.show()
```

Sample Output

```
Triangular membership values
  Temp    Cold   Comfortable     Hot   Class
  10 C    0.50          0.00    0.00   Cold
  15 C    0.25          0.50    0.00   Comfortable
  20 C    0.00          1.00    0.00   Comfortable
  25 C    0.00          0.50    0.25   Comfortable
  30 C    0.00          0.00    0.50   Hot
  35 C    0.00          0.00    0.75   Hot

Trapezoidal membership values
  Temp    Cold   Comfortable     Hot   Class
  10 C    1.00          0.00    0.00   Cold
  15 C    0.50          0.62    0.00   Comfortable
  20 C    0.00          1.00    0.00   Comfortable
  25 C    0.00          0.88    0.30   Comfortable
  30 C    0.00          0.25    0.80   Hot
  35 C    0.00          0.00    1.00   Hot

Gaussian membership values
  Temp    Cold   Comfortable     Hot   Class
  10 C    0.71          0.06    0.00   Cold
  15 C    0.25          0.38    0.00   Comfortable
  20 C    0.04          0.92    0.01   Comfortable
  25 C    0.00          0.84    0.10   Comfortable
  30 C    0.00          0.28    0.41   Hot
  35 C    0.00          0.03    0.88   Hot
(Graph window displayed - see figure below)
```



Result: Membership values of all three functions are compared; the class is the fuzzy set with the highest membership.

Exercise 1: Attendance Low / Moderate / High (values and graphs)

Program

```
import numpy as np
import matplotlib.pyplot as plt

def triangular(x, a, b, c):
    return np.maximum(np.minimum((x - a) / (b - a), (c - x) / (c - b)), 0)

def trapezoidal(x, a, b, c, d):
    return np.maximum(np.minimum(np.minimum((x - a) / (b - a), 1), (d - x) / (d - c)), 0)

def gaussian(x, mean, sigma):
    return np.exp(-((x - mean) ** 2) / (2 * sigma ** 2))

# parameters of each set: (triangular), (trapezoidal), (gaussian)
sets = {
    "Low": ((-50, 0, 50), (-10, 0, 30, 50), (0, 15)),
    "Moderate": ((25, 50, 75), (30, 50, 70, 90), (50, 12)),
    "High": ((50, 100, 150), (60, 80, 100, 110), (95, 10)),
}
functions = (triangular, trapezoidal, gaussian)
names = ("Triangular", "Trapezoidal", "Gaussian")

for i, name in enumerate(names):
    print(f"\n{name} membership values")
    print(f"{'Attendance':>11}{'Low':>8}{'Moderate':>10}{'High':>8}")
    for a in (40, 50, 60, 70, 80, 90):
        mu = [functions[i](a, *sets[s][i]) for s in sets]
        print(f"{a:>10}%{mu[0]:>8.2f}{mu[1]:>10.2f}{mu[2]:>8.2f}")

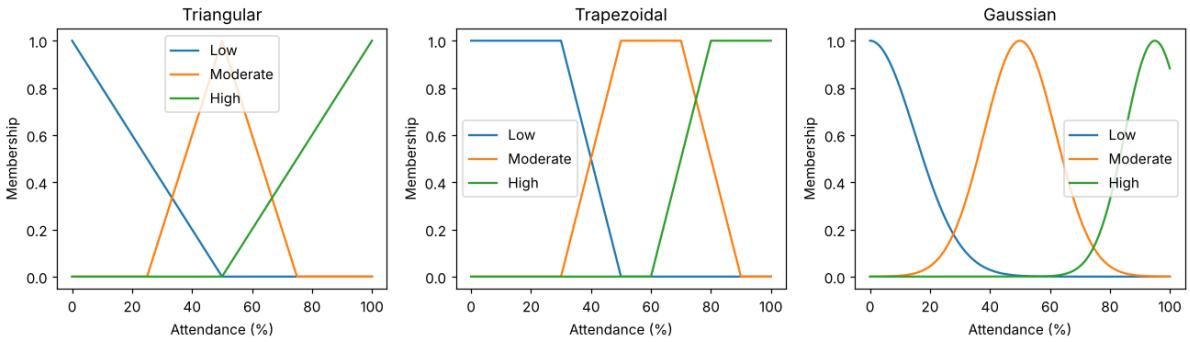
x = np.linspace(0, 100, 501)
for i, name in enumerate(names):
    plt.subplot(1, 3, i + 1)
    for s, p in sets.items():
        plt.plot(x, functions[i](x, *p[i]), label=s)
    plt.title(name)
    plt.xlabel("Attendance (%)")
    plt.ylabel("Membership")
    plt.legend()
plt.show()
```

Sample Output

```
Triangular membership values
 Attendance     Low  Moderate    High
        40%    0.20      0.60    0.00
        50%    0.00      1.00    0.00
        60%    0.00      0.60    0.20
        70%    0.00      0.20    0.40
        80%    0.00      0.00    0.60
        90%    0.00      0.00    0.80

Trapezoidal membership values
 Attendance     Low  Moderate    High
        40%    0.50      0.50    0.00
        50%    0.00      1.00    0.00
        60%    0.00      1.00    0.00
        70%    0.00      1.00    0.50
        80%    0.00      0.50    1.00
        90%    0.00      0.00    1.00

Gaussian membership values
 Attendance     Low  Moderate    High
        40%    0.03      0.71    0.00
        50%    0.00      1.00    0.00
        60%    0.00      0.71    0.00
        70%    0.00      0.25    0.04
        80%    0.00      0.04    0.32
        90%    0.00      0.00    0.88
(Graph window displayed - see figure below)
```



Result: Trapezoidal is the most suitable for High Attendance: it stays at 1 for 80-100% and never falls after the peak.

Comparison: Triangular is simple but has a single peak; Trapezoidal has a plateau (full membership over a range); Gaussian is smooth and never exactly zero. For High Attendance, Trapezoidal is best: membership stays 1 from 80% to 100%, whereas Triangular/Gaussian peak at one value and imply lower membership beyond it.

EXTRA: Additional Basic Programs (DFS, BFS and more)

Aim: Frequently asked basic programs: graph traversal (DFS, BFS), search, and neural / fuzzy basics.

BFS: Breadth First Search (graph traversal)

Program

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": [],
}

def bfs(start):
    visited = [start]
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print("Visiting", node, "| Queue now:", list(queue))
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
    return visited

print("Graph (adjacency list):", graph, "\n")
order = bfs("A")
print("\nBFS traversal order:", " -> ".join(order))
```

Sample Output

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': []}

Visiting A | Queue now: []
Visiting B | Queue now: ['C']
Visiting C | Queue now: ['D', 'E']
Visiting D | Queue now: ['E', 'F']
Visiting E | Queue now: ['F']
Visiting F | Queue now: []

BFS traversal order: A -> B -> C -> D -> E -> F
```

Result: Nodes are visited level by level using a queue.

DFS: Depth First Search (graph traversal)

Program

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": ["F"],
    "F": [],
}

def dfs(node, visited):
    visited.append(node)
    print("Visiting", node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(neighbour, visited)
    return visited

print("Graph (adjacency list):", graph, "\n")
order = dfs("A", [])
print("\nDFS traversal order:", " -> ".join(order))
```

Sample Output

```
Graph (adjacency list): {'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F'], 'D': [], 'E': ['F'], 'F': []}

Visiting A
Visiting B
Visiting D
Visiting E
Visiting F
Visiting C

DFS traversal order: A -> B -> D -> E -> F -> C
```

Result: Nodes are visited deepest-first using recursion.

Water Jug Problem using BFS

Program

```
from collections import deque

CAP_A, CAP_B, TARGET = 4, 3, 2

def next_moves(x, y):
    pour_ab = min(x, CAP_B - y)
    pour_ba = min(y, CAP_A - x)
    return [
        ("Fill A", (CAP_A, y)),
        ("Fill B", (x, CAP_B)),
        ("Empty A", (0, y)),
        ("Empty B", (x, 0)),
        ("Pour A -> B", (x - pour_ab, y + pour_ab)),
        ("Pour B -> A", (x + pour_ba, y - pour_ba)),
    ]

def bfs():
    queue = deque([[("Start", (0, 0))]])
    visited = {(0, 0)}
    while queue:
        path = queue.popleft()
        x, y = path[-1][1]
        if x == TARGET or y == TARGET:
            return path
        for action, state in next_moves(x, y):
            if state not in visited:
                visited.add(state)
                queue.append(path + [(action, state)])
    return None

path = bfs()
print("Jug A = 4 L, Jug B = 3 L, Target = 2 L\n")
print(f"{'Step':<6}{'Action':<14}{'(A, B)'}")
for i, (action, state) in enumerate(path):
    print(f"{i:<6}{action:<14}{state}")
print("\nTotal steps:", len(path) - 1)
```

Sample Output

```
Jug A = 4 L, Jug B = 3 L, Target = 2 L

Step  Action        (A, B)
0     Start         (0, 0)
1     Fill B        (0, 3)
2     Pour B -> A   (3, 0)
3     Fill B        (3, 3)
4     Pour B -> A   (4, 2)

Total steps: 4
```

Result: BFS gives the shortest sequence of steps to measure 2 litres using 4 L and 3 L jugs.

Hill Climbing Search

Program

```
def f(x):
    return -(x - 3) ** 2 + 9

x = 0
step = 0
print(f"{'Step':<6}{'x':<6}{'f(x)'}")
while True:
    print(f"{step:<6}{x:<6}{f(x)}")
    best = max((x - 1, x + 1), key=f)        # best neighbour
    if f(best) <= f(x):
        break
    x = best
    step += 1
print(f"\nPeak (local maximum) reached at x = {x}, f(x) = {f(x)}")
```

Sample Output

```
Step  x     f(x)
0     0     0
1     1     5
2     2     8
3     3     9

Peak (local maximum) reached at x = 3, f(x) = 9
```

Result: The search climbs step by step to the peak of f(x) = -(x-3)^2 + 9.

McCulloch-Pitts Neuron (AND, OR)

Program

```
inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]

def mp_neuron(x1, x2, w1, w2, theta):
    net = x1 * w1 + x2 * w2
    return net, 1 if net >= theta else 0

for gate, theta in (("AND", 2), ("OR", 1)):
    print(f"\n{gate} gate (w1 = w2 = 1, threshold = {theta})")
    print("x1  x2  net  y")
    for x1, x2 in inputs:
        net, y = mp_neuron(x1, x2, 1, 1, theta)
        print(f"{x1}   {x2}   {net}    {y}")
```

Sample Output

```
AND gate (w1 = w2 = 1, threshold = 2)
x1  x2  net  y
0   0   0    0
0   1   1    0
1   0   1    0
1   1   2    1

OR gate (w1 = w2 = 1, threshold = 1)
x1  x2  net  y
0   0   0    0
0   1   1    1
1   0   1    1
1   1   2    1
```

Result: Fixed-weight McCulloch-Pitts neurons reproduce the AND and OR truth tables.

Perceptron Learning (AND gate)

Program

```
inputs = [(0, 0), (0, 1), (1, 0), (1, 1)]
targets = [0, 0, 0, 1]
w1 = w2 = bias = 0
lr = 1

def output(x1, x2):
    return 1 if w1 * x1 + w2 * x2 + bias > 0 else 0

for epoch in range(1, 11):
    errors = 0
    for (x1, x2), t in zip(inputs, targets):
        e = t - output(x1, x2)
        if e != 0:
            w1 += lr * e * x1
            w2 += lr * e * x2
            bias += lr * e
            errors += 1
    print(f"Epoch {epoch}: w1 = {w1}, w2 = {w2}, bias = {bias}, errors = {errors}")
    if errors == 0:
        break

print("\nTesting the trained perceptron")
print("x1  x2  Target  Output")
for (x1, x2), t in zip(inputs, targets):
    print(f"{x1}   {x2}     {t}       {output(x1, x2)}")
```

Sample Output

```
Epoch 1: w1 = 1, w2 = 1, bias = 1, errors = 1
Epoch 2: w1 = 2, w2 = 1, bias = 0, errors = 3
Epoch 3: w1 = 2, w2 = 1, bias = -1, errors = 3
Epoch 4: w1 = 2, w2 = 2, bias = -1, errors = 2
Epoch 5: w1 = 2, w2 = 1, bias = -2, errors = 1
Epoch 6: w1 = 2, w2 = 1, bias = -2, errors = 0

Testing the trained perceptron
x1  x2  Target  Output
0   0     0       0
0   1     0       0
1   0     0       0
1   1     1       1
```

Result: The perceptron learns weights that classify the AND gate correctly.

Hebb Network (AND gate, bipolar)

Program

```
inputs = [(1, 1), (1, -1), (-1, 1), (-1, -1)]
targets = [1, -1, -1, -1]
w1 = w2 = bias = 0

print("x1  x2   t   dw1 dw2  db | w1  w2   b")
for (x1, x2), t in zip(inputs, targets):
    dw1, dw2, db = x1 * t, x2 * t, t
    w1, w2, bias = w1 + dw1, w2 + dw2, bias + db
    print(f"{x1:>2} {x2:>3} {t:>3} {dw1:>5} {dw2:>3} {db:>3} | {w1:>2} {w2:>3} {bias:>3}")

print("\nTesting the trained network")
for (x1, x2), t in zip(inputs, targets):
    y = 1 if w1 * x1 + w2 * x2 + bias > 0 else -1
    print(f"Input ({x1:>2}, {x2:>2}) -> Output {y:>2} (target {t:>2})")
```

Sample Output

```
x1  x2   t    dw1 dw2  db | w1  w2   b
 1   1   1     1   1   1 |  1   1   1
 1  -1  -1    -1   1  -1 |  0   2   0
-1   1  -1     1  -1  -1 |  1   1  -1
-1  -1  -1     1   1  -1 |  2   2  -2

Testing the trained network
Input ( 1,  1) -> Output  1 (target  1)
Input ( 1, -1) -> Output -1 (target -1)
Input (-1,  1) -> Output -1 (target -1)
Input (-1, -1) -> Output -1 (target -1)
```

Result: Hebb rule weights classify the bipolar AND gate correctly.

Fuzzy Set Operations

Program

```
A = {"x1": 0.2, "x2": 0.5, "x3": 0.8, "x4": 1.0}
B = {"x1": 0.6, "x2": 0.3, "x3": 0.7, "x4": 0.1}

print(f"{'Element':<8}{'A':<6}{'B':<6}{'Union':<7}{'Inter':<7}{'not A':<7}not B")
for x in A:
    u, i = max(A[x], B[x]), min(A[x], B[x])
    print(f"{x:<8}{A[x]:<6}{B[x]:<6}{u:<7}{i:<7}{1 - A[x]:<7.2f}{1 - B[x]:.2f}")
```

Sample Output

```
Element A     B     Union  Inter  not A  not B
x1      0.2   0.6   0.6    0.2    0.80   0.40
x2      0.5   0.3   0.5    0.3    0.50   0.70
x3      0.8   0.7   0.8    0.7    0.20   0.30
x4      1.0   0.1   1.0    0.1    0.00   0.90
```

Result: Union (max), intersection (min) and complement (1 - x) of two fuzzy sets.